

Contents

1	Auto-encoding variational Bayes	2
1.1	Preliminary	2
1.1.1	Monte Carlo gradient estimator	2
1.1.2	Evidence lower bound (ELBO)	2
1.2	Method	3
1.2.1	Auto-encoding variational Bayes algorithm	3
1.2.2	Variational Auto-encoders	3
2	Neural ODE	4
2.1	Method	4
2.1.1	The adjoint and two equations	5
2.1.2	Backward propagation of neural ODE	6
3	Continuous-Time Latent ODE Graph VAE	7
3.1	Method	7
3.2	Experiment	11
3.2.1	Learning from smoothness	11
3.2.2	Restricted expression ability	12

1 Auto-encoding variational Bayes

Auto-encoding variational Bayes method [2] is designed for problems with latent variables and intractable distributions so that EM and variational inference do not work. The method uses generative models to create variational posterior and optimize model parameters by maximizing Evidence Lower Bound (ELBO). Hence it provides a framework for combining neural networks and Bayesian inference.

1.1 Preliminary

1.1.1 Monte Carlo gradient estimator

Suppose we have a random variable $z \sim q_\phi(z)$ where ϕ is the parameter of the density q . For an arbitrary function $f \in L_2(q_\phi)$, we want to estimate the gradient of $\mathbb{E}_{q_\phi(z)}[f(z)]$ w.r.t ϕ . We have two approaches that use Monte Carlo method to approximate the expectation.

$$\nabla_\phi \mathbb{E}_{q_\phi(z)}[f(z)] = \mathbb{E}_{q_\phi(z)}[f(z) \nabla_\phi \log q_\phi(z)] \approx \frac{1}{N} \sum_{i=1}^N f(z_i) \nabla_\phi \log q_\phi(z_i). \quad (1)$$

Suppose $z = g(\phi, \epsilon)$, $\epsilon \sim p(\epsilon)$. Then $p(\epsilon) = q_\phi(g(\phi, \epsilon)) \frac{\partial g(\phi, \epsilon)}{\partial \epsilon}$.

$$\nabla_\phi \mathbb{E}_{q_\phi(z)}[f(z)] = \mathbb{E}_\epsilon[\nabla_\phi f(g(\phi, \epsilon))] \approx \frac{1}{N} \sum_{i=1}^N \nabla_\phi f(g(\phi, \epsilon_i)). \quad (2)$$

Method (1) uses score function $\nabla_\phi \log q_\phi(z)$ and (2) uses reparameterization. The score-function approach is unbiased but with high variance. The reparameterization approach has low variance but requires a differentiable pathway.

1.1.2 Evidence lower bound (ELBO)

Suppose the observable variable X and the latent variable Z belong to the same parametric family p_θ , $\theta \in \Theta \subseteq \mathbb{R}^d$. To estimate the true posterior $p_\theta(z|x)$, we use maximum likelihood method to optimize the parameter θ . However when the true marginal density $p_\theta(x) = \int p_\theta(x|z)p_\theta(z)dz$ is intractable, by Bayes formula, $p_\theta(z|x) = \frac{p_\theta(x|z)p_\theta(z)}{p_\theta(x)}$ is also intractable. This makes the gradient optimizer lose its power. One optional approach is to use variational density $q_\phi(z|x)$ to approximate the true posterior. And the variational posterior is optimized by maximizing the ELBO.

$$\begin{aligned} \log p_\theta(x) &= \int \log p_\theta(x) q_\phi(z|x) dz \\ &= \int (\log p_\theta(z, x) - \log p_\theta(z|x)) q_\phi(z|x) dz \\ &= \int (\log p_\theta(z, x) - \log q_\phi(z|x)) q_\phi(z|x) dz + \int (\log q_\phi(z|x) - \log p_\theta(z|x)) q_\phi(z|x) dz \\ &= \mathcal{L}(\theta, \phi; x) + D_{KL}(q_\phi(z|x) || p_\theta(z|x)). \end{aligned} \quad (3)$$

By (3), since Kullback-Leibler divergence is nonnegative and $\log p_\theta(x)$ is a constant irrelevant to ϕ , we can maximize the ELBO $\mathcal{L}(\theta, \phi; x)$ to minimize the KL divergence between the variational posterior and the true posterior.

Another expression of ELBO is given by (4).

$$\begin{aligned}
\mathcal{L}(\theta, \phi; x) &= \int (\log p_\theta(z, x) - \log q_\phi(z|x)) q_\phi(z|x) dz \\
&= \int (\log p_\theta(x|z) + \log p_\theta(z) - \log q_\phi(z|x)) q_\phi(z|x) dz \\
&= \int \log p_\theta(x|z) q_\phi(z|x) dz - \int (\log q_\phi(z|x) - \log p_\theta(z)) q_\phi(z|x) dz \\
&= \mathbb{E}_{q_\phi(z|x)} [\log p_\theta(x|z)] - D_{KL}(q_\phi(z|x) || p_\theta(z)).
\end{aligned} \tag{4}$$

1.2 Method

The Auto-encoding variational Bayes algorithm uses arbitrary encoder to encode data to posterior function (parameters). The latent state (embedding) is sampled from the variational posterior. The model learnable parameters are updated using the gradient computed from the ELBO. The algorithm not only gives freedom to choose encoder and decoder but also provides opportunity to combine other deterministic methods to embed data to latent state variable.

1.2.1 Auto-encoding variational Bayes algorithm

From (3) and (4), we have two expressions for the ELBO.

$$\mathcal{L}(\theta, \phi; x) = \begin{cases} \mathbb{E}_{q_\phi(z|x)} [\log p_\theta(z, x) - \log q_\phi(z|x)] \\ \mathbb{E}_{q_\phi(z|x)} [\log p_\theta(x|z)] - D_{KL}(q_\phi(z|x) || p_\theta(z)) \end{cases} \tag{5}$$

For any differentiable pathway g_ϕ and auxiliary noise variable $\epsilon \sim p(\epsilon)$, we apply the reparameterization trick to (5) to get the SGVBs.

For an observation $X = x$, we let $Z = g_\phi(\epsilon, x)$. Then the i.i.d. Monte Carlo sample $z_1, \dots, z_N \sim q_\phi(z|x)$ are determined by i.i.d. $\epsilon_1, \dots, \epsilon_N \sim p(\epsilon)$. And the ELBO is approximated by

$$\tilde{\mathcal{L}}(\theta, \phi; x) = \begin{cases} \frac{1}{N} \sum_{i=1}^N (\log p_\theta(z_i, x) - \log q_\phi(z_i|x)) \\ \frac{1}{N} \sum_{i=1}^N \log p_\theta(x|z_i) - D_{KL}(q_\phi(z_i|x) || p_\theta(z_i)) \end{cases} \tag{6}$$

One can take any of the formations to approximate the ELBO. Since the KL divergence usually can be analytically computed, we don't need to approximate it with Monte Carlo method.

The Auto-encoding variational Bayes algorithm (Minibatch version) is given as follows.

Algorithm 1 Auto-encoding Variational Bayes

```

Initialize  $\theta^{(0)}$ 
for  $k = 1 : K$  do
   $\phi^{(k)} \leftarrow G_{\theta^{(k)}}(X)$ 
  sample  $\epsilon \sim p(\epsilon)$ 
   $z_k \leftarrow g_{\phi^{(k)}}(\epsilon)$ 
  compute gradient
  update  $\theta$ 
end for
return parameters

```

1.2.2 Variational Auto-encoders

The Auto-encoding variational Bayes algorithm provides freedom for choosing the variational posterior $q_\phi(z|x)$. When the variational posterior is determined by a neural network, the algorithm is equivalent to an auto-encoder which uses a probabilistic encoder $q_\phi(z|x)$.

As an example, we assume the posterior belongs to the Gaussian family, i.e. $q_\phi(z|x) = \mathcal{N}(\mu_x, \sigma_x^2)$. The parameters μ_x and σ_x^2 are mapped from observation x by MLPs with parameters $\phi = (W_0, W_1, W_2, b_0, b_1, b_2)$. This process is defined as follows.

$$\begin{aligned} h_x &= \tanh(W_0 x + b_0) \\ \mu_x &= W_1 h_x + b_1 \\ \log \sigma_x^2 &= W_2 h_x + b_2 \end{aligned}$$

Let the auxiliary random variable $\epsilon \sim p(\epsilon) = \mathcal{N}(0, 1)$. Then the differentiable pathway g_ϕ is given by

$$\forall z \sim q_\phi(z|x), \exists \epsilon \sim p(\epsilon), \text{ s.t. } z = g_\phi(x, \epsilon) = \mu_x + \sigma_x \epsilon.$$

The probabilistic decoding process is also determined by a MLPs with the same structure but different hyperparameters $\theta = (w_3, w_4, w_5, b_3, b_4, b_5)$.

$$\begin{aligned} h_z &= \tanh(W_3 z + b_3) \\ \mu_z &= W_4 h_z + b_4 \\ \log \sigma_z^2 &= W_5 h_z + b_5 \end{aligned}$$

We sample the prediction x_{pred} from $p_\theta(x|z) = \mathcal{N}(\mu_z, \sigma_z^2)$.

The non-probabilistic decoder can be an activation function operating on the inner product of embeddings.

$$x_{pred} = \sigma(\langle z, z \rangle).$$

2 Neural ODE

The neural ODE [1] substitutes the discrete neural network hidden layers by a differentiable function. It is a smooth version of residual neural networks. The output of this continuous-depth network is the solution of an initial value point ODE problem ending at an arbitrarily depth. The forward propagation does not store the gradients and intermediate values and is conducted by a black-box ODE solver. The neural ODE based models have constant memory cost and can trade numerical precision with time cost.

2.1 Method

The continuous-depth network is described by an initial value point problem with an invariant network function f , as shown below. We suppose the output of latent state z and network function f are all vectors.

$$\begin{cases} \frac{dz(t)}{dt} = f(z(t), t, \theta) \\ z(t_0) = z_0 \end{cases}$$

The output at time t_1 is numerically solved by a black-box ODE solver, for example Runge-Kutta method. Analytically, we have the solution given by

$$z(t_1) = z(t_0) + \int_{t_0}^{t_1} f(z(t), t, \theta) dt.$$

The corresponding loss of the model at time t_1 is

$$L(z(t_1)) = L\left(z(t_0) + \int_{t_0}^{t_1} f(z(t), t, \theta) dt\right).$$

The gradient w.r.t the model parameter θ is calculated by

$$\begin{aligned} \frac{\partial L}{\partial \theta} &= \left(\frac{\partial z(t_1)}{\partial \theta}\right)^T \frac{\partial L}{\partial z(t_1)} = \left(\frac{\partial z(t_0)}{\partial \theta} + \frac{\partial}{\partial \theta} \int_{t_0}^{t_1} f(z(t), t, \theta) dt\right)^T \frac{\partial L}{\partial z(t_1)} \\ &= \left(\frac{\partial z(t_0)}{\partial \theta} + \int_{t_0}^{t_1} \frac{\partial f}{\partial \theta} + \frac{\partial f}{\partial z(t)} \frac{\partial z(t)}{\partial \theta} dt\right)^T \frac{\partial L}{\partial z(t_1)}, \end{aligned}$$

where the gradients $\frac{\partial L}{\partial z(t)}$ and $\frac{\partial f}{\partial z(t)}$ can be calculated by auto-differentiation since f is clearly known. However, $\frac{\partial z(t)}{\partial \theta}$ is intractable since the latent dynamic $z(t)$ is unobservable.

To solve this, we introduce the adjoint sensitivity method to get the trajectory of $\frac{\partial L}{\partial z(t)}$ by solving another ODE. This helps removing the problematic gradients in computing $\frac{\partial L}{\partial \theta}$.

2.1.1 The adjoint and two equations

We define the adjoint by $a(t) := \frac{\partial L}{\partial z(t)}$. The adjoint satisfies two equations

$$\frac{da(t)}{dt} = - \left(\frac{\partial}{\partial z(t)} f(z(t), t, \theta) \right)^T a(t), \quad (7)$$

$$\frac{\partial L}{\partial \theta} = - \int_{t_1}^{t_0} \left(\frac{\partial f}{\partial \theta} \right)^T a(t) dt, \quad (8)$$

where the gradient $\frac{\partial f}{\partial \theta}$ in (8) is the derivative of $f(z(t), t, \theta)$ w.r.t θ treating z as constant. The two equations record the process of calculating the prior knowledge for updating the parameter. (8) depends on the trajectory of $a(t)$ which means we need to simultaneously compute the value of $a(t)$ when calculating the numerical approximation of the integral. Before understanding the full backward propagation process, we give the proofs of the two equations respectively.

Proof of (7) The proof relies on using the definition of derivative as t_1 and t_0 are arbitrarily close. We start by computing the derivative of loss about $z(t_0)$.

Suppose $a(t) := \frac{\partial L}{\partial z(t)}$ is differentiable about t . We consider a scalar-valued loss L .

$$\begin{aligned} \frac{\partial L}{\partial z(t_0)} &= \left(\frac{\partial z(t_1)}{\partial z(t_0)} \right)^T \frac{\partial L}{\partial z(t_1)} \\ &= \left(I + \frac{\partial}{\partial z(t_0)} \int_{t_0}^{t_1} f(z(t), t, \theta) dt \right)^T \frac{\partial L}{\partial z(t_1)} \\ &= \frac{\partial L}{\partial z(t_1)} + \left(\frac{\partial}{\partial z(t_0)} \int_{t_0}^{t_1} f(z(t), t, \theta) dt \right)^T \frac{\partial L}{\partial z(t_1)} \\ \frac{\partial L}{\partial z(t_1)} - \frac{\partial L}{\partial z(t_0)} &= - \left(\frac{\partial}{\partial z(t_0)} \int_{t_0}^{t_1} f(z(t), t, \theta) dt \right)^T \frac{\partial L}{\partial z(t_1)} \\ \frac{\frac{\partial L}{\partial z(t_1)} - \frac{\partial L}{\partial z(t_0)}}{t_1 - t_0} &= - \frac{1}{t_1 - t_0} \left(\frac{\partial}{\partial z(t_0)} \int_{t_0}^{t_1} f(z(t), t, \theta) dt \right)^T \frac{\partial L}{\partial z(t_1)} \\ \lim_{t_1 \rightarrow t_0} \frac{\frac{\partial L}{\partial z(t_1)} - \frac{\partial L}{\partial z(t_0)}}{t_1 - t_0} &= - \lim_{t_1 \rightarrow t_0} \frac{1}{t_1 - t_0} \left(\frac{\partial}{\partial z(t_0)} \int_{t_0}^{t_1} f(z(t), t, \theta) dt \right)^T \frac{\partial L}{\partial z(t_1)} \\ \left. \frac{\partial}{\partial t} \frac{\partial L}{\partial z(t)} \right|_{t=t_0} &= - \left(\frac{\partial}{\partial z(t_0)} f(z(t_0), t, \theta) \right)^T \frac{\partial L}{\partial z(t_0)} \\ \left. \frac{da(t)}{dt} \right|_{t=t_0} &= - \left(\frac{\partial}{\partial z(t_0)} f(z(t_0), t, \theta) \right)^T a(t_0). \end{aligned}$$

By the randomness of t_0 , we have

$$\frac{da(t)}{dt} = - \left(\frac{\partial}{\partial z(t)} f(z(t), t, \theta) \right)^T a(t).$$

Proof of (8) Aside from the definition of derivative, the proof relies on the trajectory of adjoint given by the Equation 7. We also need the intermediate expression of the gradient $\frac{\partial L}{\partial \theta}$. We suppose the parameter $\theta \in \mathbb{R}^d$.

$$\begin{aligned}
\frac{\partial L}{\partial \theta} &= \left(\frac{\partial z(t_1)}{\partial \theta} \right)^T a(t_1) = \left(\frac{\partial z(t_0)}{\partial \theta} + \frac{\partial}{\partial \theta} \int_{t_0}^{t_1} f(z(t), t, \theta) dt \right)^T a(t_1) \\
&= \left(\frac{\partial z(t_0)}{\partial \theta} + \int_{t_0}^{t_1} \frac{\partial f}{\partial \theta} + \frac{\partial f}{\partial z(t)} \frac{\partial z(t)}{\partial \theta} dt \right)^T a(t_1).
\end{aligned}$$

From the above deduction, we have a new equation.

$$\begin{aligned}
\left(\frac{\partial z(t_1)}{\partial \theta} \right)^T a(t_1) &= \left(\frac{\partial z(t_0)}{\partial \theta} + \int_{t_0}^{t_1} \frac{\partial f}{\partial \theta} + \frac{\partial f}{\partial z(t)} \frac{\partial z(t)}{\partial \theta} dt \right)^T a(t_1) \\
\left(\frac{\partial z(t_1)}{\partial \theta} - \frac{\partial z(t_0)}{\partial \theta} \right)^T a(t_1) &= \left(\int_{t_0}^{t_1} \frac{\partial f}{\partial \theta} + \frac{\partial f}{\partial z(t)} \frac{\partial z(t)}{\partial \theta} dt \right)^T a(t_1) \\
\lim_{t_1 \rightarrow t_0} \frac{1}{t_1 - t_0} \left(\frac{\partial z(t_1)}{\partial \theta} - \frac{\partial z(t_0)}{\partial \theta} \right)^T a(t_1) &= \lim_{t_1 \rightarrow t_0} \frac{1}{t_1 - t_0} \left(\int_{t_0}^{t_1} \frac{\partial f}{\partial \theta} + \frac{\partial f}{\partial z(t)} \frac{\partial z(t)}{\partial \theta} dt \right)^T a(t_1) \\
\left(\frac{\partial}{\partial t} \frac{\partial z(t)}{\partial \theta} \right)^T \Big|_{t=t_0} a(t_0) &= \left(\frac{\partial f}{\partial \theta} + \frac{\partial f}{\partial z(t)} \frac{\partial z(t)}{\partial \theta} \right)^T \Big|_{t=t_0} a(t_0).
\end{aligned}$$

By the randomness of t_0 , we have

$$\begin{aligned}
\left(\frac{\partial}{\partial t} \frac{\partial z(t)}{\partial \theta} \right)^T a(t) &= \left(\frac{\partial f}{\partial \theta} + \frac{\partial f}{\partial z(t)} \frac{\partial z(t)}{\partial \theta} \right)^T a(t) = \left(\frac{\partial f}{\partial \theta} \right)^T a(t) + \left(\frac{\partial f}{\partial z(t)} \frac{\partial z(t)}{\partial \theta} \right)^T a(t) \\
&= \left(\frac{\partial f}{\partial \theta} \right)^T a(t) + \left(\frac{\partial z(t)}{\partial \theta} \right)^T \left(\frac{\partial f}{\partial z(t)} \right)^T a(t) \\
&= \left(\frac{\partial f}{\partial \theta} \right)^T a(t) - \left(\frac{\partial z(t)}{\partial \theta} \right)^T \frac{da(t)}{dt} \quad (\text{By (7)}) \\
\left(\frac{\partial}{\partial t} \frac{\partial z(t)}{\partial \theta} \right)^T a(t) + \left(\frac{\partial z(t)}{\partial \theta} \right)^T \frac{da(t)}{dt} &= \left(\frac{\partial f}{\partial \theta} \right)^T a(t) \\
\frac{d}{dt} \left(\left(\frac{\partial z(t)}{\partial \theta} \right)^T a(t) \right) &= \left(\frac{\partial f}{\partial \theta} \right)^T a(t) \\
\left(\frac{\partial z(t_1)}{\partial \theta} \right)^T a(t_1) - \left(\frac{\partial z(t_0)}{\partial \theta} \right)^T a(t_0) &= \int_{t_0}^{t_1} \left(\frac{\partial f}{\partial \theta} \right)^T a(t) dt \\
\left(\frac{\partial z(t_1)}{\partial \theta} \right)^T a(t_1) &= \int_{t_0}^{t_1} \left(\frac{\partial f}{\partial \theta} \right)^T a(t) dt.
\end{aligned}$$

The last step is by the fact that the initial layer $z(t_0)$ is irrelevant to the parameter θ . Since the LHS is just the gradient $\frac{\partial L}{\partial \theta}$, we have

$$\frac{\partial L}{\partial \theta} = - \int_{t_1}^{t_0} \left(\frac{\partial f}{\partial \theta} \right)^T a(t) dt.$$

2.1.2 Backward propagation of neural ODE

To get the gradient $\frac{\partial L}{\partial \theta}$, we compute the integral (8) and need the simultaneous computation of the adjoint $a(t)$ through (7). Among the two processes, the gradients $\frac{\partial f}{\partial z(t)}$ and $\frac{\partial f}{\partial \theta}$ are calculated by auto-differentiation and the vector-Hessian products are computed subsequently. The adjoint trajectory ODE (7) is solved by a random ODE-solver starting from the initial time t_1 and initial value $a(t_1) = \frac{\partial L}{\partial z(t_1)}$. At each end of a step, the output $a(t_{\text{intermediate}})$ is stored temporarily and accumulated by the integral. This proceeds until t_0 , making sure the memory cost is a constant.

3 Continuous-Time Latent ODE Graph VAE

Suppose we observe a sequence of graphs at irregular times from a single dynamic system, for example the traffic flow of a city’s transportation system, where each edge in the adjacency matrix is the traffic flow from one hub to the other, and the node feature shows the population and other physical conditions. The sequence can also represent an individual trajectory, like the brain connectivities scanned during a compact time period, and cell-cell communication recorded from some blood samples collected from a single subject within days. In a word, the sequence we consider in this case study must be a series of trajectories showing subject-wise evolution within a consecutive period to make sure causally consistent adjacency transitions with temporal differentiability¹. Exclusion without the features we require includes brain connectivities recorded from a group with different ages. This kind of data is extremely noised by violating the single subject requirement, and is not temporally continuous since the gap between different ages is very large.

To facilitate the mathematical illustration of our model, we denote the time series by

$$\mathcal{D} = \{(t_1, X_{t_1}, A_{t_1}), \dots, (t_T, X_{t_T}, A_{t_T})\},$$

where $A_t \in \mathbb{R}^{N \times N}$ is a weighted adjacency matrix and $X_t \in \mathbb{R}^{N \times d}$ is a node feature matrix. The goal is to learn a continuous-time latent dynamical system capable of encoding graph evolution and predicting future adjacency matrices.

With this purpose, we propose the Continuous-Time Latent ODE Graph VAE, which is constructed under the variational autoencoder framework. The model is featured by a latent state denoted by z . It is a $N \times d_z$ real matrix whose rows $z_1, \dots, z_N$ are embeddings of nodes and corresponding features. The learning procedure is given in the next subsection.

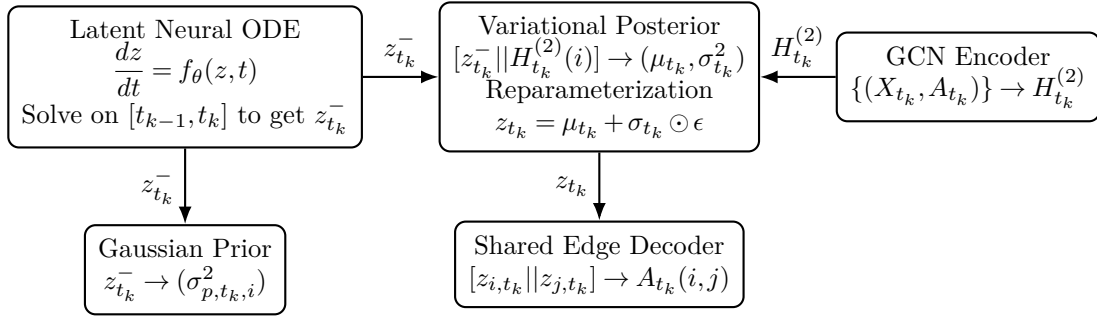


Figure 1: Architecture of the Continuous-Time Latent ODE Graph VAE. A GCN encodes graph snapshots, the latent state evolves via a Neural ODE, prior and posterior MLPs define Gaussian distributions on node embeddings, and a shared edge decoder reconstructs adjacency matrices.

3.1 Method

We briefly introduce the learning steps before mathematical illustration. The forward propagation starts first by applying a GCN encoder to the original graph data to embed the weighted adjacency matrix and its node features into a low dimensional graph representation. Then it uses neural ODE to update the latent state by solving a initial valued problem. The network function to be integrated consists of a 2-layer MLP using the latent state and time as input variables. The output of the neural ODE is the prior mean of the latent state at the next time stamp. The prior distribution of latent state at the next time stamp is learnt by a 2-layer MLP using this prior mean and time gap between the current and the next. The variational posterior is learnt by another 2-layer MLP using the prior mean and low dimensional graph representation. Both the prior and variational posterior are assumed to be joint products of independent Gaussian distributions of z_i . The joint edge decoder for edge weights can be factorized into a joint product of edge decoders for each edge weight. It is also Gaussian and learnt by using latent state pairs (z_i, z_j) sampled from variational posterior. The backward propagation uses the accumulated gradients of model parameters to maximize the ELBO with adam optimizer.

¹Differentiability is rough here but learnable by algorithm. We say very small time gap between two proximal data points will contribute to temporal differentiability.

To make the model training stable, we constrain the operator norm of the linear coefficients in the MLPs to make the latent ODE Lipschitz-continuous and hence avoid the neural ODE falling into a stiff situation. Moreover, if the KL term is too strong early in training, the encoder may learn to output a posterior that matches the prior regardless of the input. This makes the latent variables uninformative. Hence, we employ KL annealing to alleviate posterior collapse.

GCN Encoder At each time t , we encode the graph $G_t = (X_t, A_t)$ using a two-layer GCN:

$$\begin{aligned} H_t^{(1)} &= \text{ReLU}(\tilde{D}_t^{-\frac{1}{2}} \tilde{A}_t \tilde{D}_t^{-\frac{1}{2}} X_t W_1^G), \\ H_t^{(2)} &= \tilde{D}_t^{-\frac{1}{2}} \tilde{A}_t \tilde{D}_t^{-\frac{1}{2}} H_t^{(1)} W_2^G, \end{aligned}$$

where $\tilde{A}_t = A_t + I$, and $\tilde{D}_t$ is degree matrix of $\tilde{A}_t$. W_1^G and W_2^G are model parameters to be learnt. The output $H_t^{(2)} \in \mathbb{R}^{N \times d_h}$ contains node-wise embeddings.

Node-wise Latent Neural ODE Each node i has a time-continuous latent trajectory $z_i(t) \in \mathbb{R}^{d_z}$. We define the ODE

$$\frac{dz(t)}{dt} = f_\theta(z(t), t),$$

with $z(t) = \begin{bmatrix} z_1(t) \\ \vdots \\ z_N(t) \end{bmatrix} \in \mathbb{R}^{N \times d_z}$. To ensure stability, the vector field uses: (i) Lipschitz-constrained linear layers (spectral normalization), (ii) smooth activations (SoftPlus), and (iii) learnable scaling coefficient. All these contribute to a MLP network function

$$f_\theta(z_i(t), t) = s \cdot W_1^O \text{SoftPlus}([z_i(t) || t] W_0^O + b_0^O) + b_1^O, \quad (9)$$

where s is a learnable scaling term, and the norm of W_1^O and W_2^O are controlled via spectral normalization. This MLP maps from $\mathbb{R}^{N \times (d_z + 1)}$ to $\mathbb{R}^{N \times d_z}$. Of note, although the three measures maintain the stability, they also force (9) to have a slowly varying trajectory by restricted to low-curvature and Lipschitz continuous flows. This result in the liability in learning the nonlinear and discontinuous dynamic.

As proposed by Chen et al. [1], we use a black box ODE solver to get the value of latent state at the end of an interval. Given a interval $[t_{k-1}, t_k]$, it solves $z(t_k)$ from an initial value problem

$$\begin{cases} \frac{dz(t)}{dt} = f_\theta(z(t), t), \\ z(t_{k-1}) = z_{t_{k-1}}. \end{cases}$$

We denote solution $z(t_k)$ by $z_{t_k}^-$, which is the prior mean.

We set the initial value of the ODE system on first time interval by a linear layer using input as the GCN layer processed embedding $H_{t_0}^{(2)}$:

$$z_{t_0} = H_{t_0}^{(2)} W + b,$$

where W and b are randomly initialized.

It's worth mentioning that the network function (9) is Markovian with first-order temporal dependence, since the latent variable is updated only by the current time and nearest previous latent state. This greatly decrease the ability to leverage the past time information and subsequently lead to a short-term temporal memory system. In this way, the algorithm only excels at learning near-equilibrium or slowly drifting dynamics. Furthermore, the latent state is developed only on the GCN aggregated information at the beginning of the time series. The information from nodes and edges on the intermediate time stamps are not included. This implies the algorithm can only performs well on the situations where node-edge interactions are mostly static or weak, i.e. the information flows in an approach invariant to both time and topology of the network.

Gaussian Prior We suppose the latent states $z_1, \dots, z_N$ for each node are independent. Given the ODE prediction $z_{t_k}^- \in \mathbb{R}^{N \times d_z}$, the prior is centered around it,

$$p(z_{t_k}) = \prod_{i=1}^N \mathcal{N}(z_{i,t_k} \mid z_{i,t_k}^-, \text{diag}(\sigma_{p,t_k,i}^2)),$$

with variance

$$\sigma_{p,t_k,i}^2 = \text{SoftPlus}(W_1^p \tanh([z_{i,t_k}^- \parallel \log(1+t_k-t_{k-1})]W_0^p + b_0^p) + b_1^p).$$

This MLP is shared cross all node level latent states and maps from $\mathbb{R}^{d_z+1}$ to $\mathbb{R}^{d_z}$.

Variational Posterior For each time stamp t_k , the posterior factorizes over nodes and is a joint distribution of N multivariate normal distributions

$$q(z_{t_k}) = \prod_{i=1}^N \mathcal{N}(z_{i,t_k} \mid \mu_{i,t_k}, \text{diag}(\sigma_{i,t_k}^2)).$$

We condition on both the ODE prior state and the GCN embedding:

$$\begin{cases} \mu_{i,t_k} = W_1^q \tanh(W_0[z_{i,t_k}^- \parallel H_{t_k}^{(2)}(i)] + b_0) + b_1^q, \\ \log(\sigma_{i,t_k}^2) = W_1^s \tanh(W_0[z_{i,t_k}^- \parallel H_{t_k}^{(2)}(i)] + b_0) + b_1^s. \end{cases}$$

The two MLPs are shared cross node level latent states and map from $\mathbb{R}^{d_z+d_h}$ to $\mathbb{R}$ and $\mathbb{R}^{d_z}$ respectively.

Of note, both the prior and posterior are based on the mean field assumption that the joint distribution can be factorized by separated marginals, which can be seen by the diagonal variances. Subsequently, the algorithm can not catch correlated node dynamics and hence can not represent the global latent.

Shared Edge Decoder we use a Gaussian decoder to get the weighted adjacency matrix A_{t_k} from the latent dynamic z_{t_k} sampled from $q(z_{t_k})$ by reparameterization $z_{t_k} = \mu_{t_k} + \sigma_{t_k} \odot \epsilon$, $\epsilon \sim \mathcal{MN}(0, I_N, I_{d_z})$. Instead of a separate MLP for each edge, we use a single shared decoder:

$$p(A_{t_k}(i,j) \mid z_{i,t_k}, z_{j,t_k}) = \mathcal{N}(\mu_{ij,t_k}, \sigma_d^2),$$

where

$$\begin{cases} \mu_{ij,t_k} = W_1^d \tanh([z_{i,t_k} \parallel z_{j,t_k}]W_0^d + b_0^d) + b_1^d, \\ \sigma_d^2 = 1. \end{cases}$$

The MLP maps from $\mathbb{R}^{2d_z}$ to $\mathbb{R}$.

When the adjacency matrix is binary, we use a Bernoulli decoder, given by

$$p(A_{t_k}(i,j) \mid z_{i,t_k}, z_{j,t_k}) = p^{A_{t_k}(i,j)}(1-p)^{1-A_{t_k}(i,j)},$$

where

$$\begin{cases} \text{logit} = W_1^d \tanh([z_{i,t_k} \parallel z_{j,t_k}]W_0^d + b_0^d) + b_1^d, \\ p = \text{Sigmoid}(\text{logit}). \end{cases}$$

It is clear that a shared edge decoder for all edges will enforce the prediction rule to be homogeneous, and make the predicted graph share a similar pattern with the latent space. As a consequence, the algorithm performs bad when the true graph dynamic is beyond its foreseeing but still converges steadily. Moreover, similar to the design of network function (9), the decoder is static and memoryless by the lack of the information from past edges and latent state representing edges. Such a decoder will infer the future edges by the latent state of nodes and current time but without considering the edge itself. This implies the decoder favors smooth edge evolution even when incorrect.

Evidence Lower Bound (ELBO) The per-time ELBO is

$$\mathcal{L}_k = \mathbb{E}_q[\log p(A_{t_k} | z_{t_k})] - \beta_k D_{\text{KL}}(q(z_{t_k}) \| p(z_{t_k})),$$

where β_k is a KL annealing coefficient increasing from 0 to 1.

The KL divergence has a closed-form expression, which is given by

$$\begin{aligned} D_{\text{KL}}(q||p) &= \mathbb{E}_q \left[\log \frac{\prod_{i=1}^{d_z} \mathcal{N}(\mu_{i,t_k}, \text{diag}(\sigma_{i,t_k}^2))}{\prod_{i=1}^{d_z} \mathcal{N}(z_{i,t_k}^-, \text{diag}(\sigma_{i,p,t_k}^2))} \right] \\ &= \frac{1}{2} \sum_{i=1}^N \left\{ \log \prod_{j=1}^{d_z} \frac{\sigma_{j,i,p,t_k}^2}{\sigma_{j,i,t_k}^2} - \mathbb{E}_q \left[\sum_{j=1}^{d_z} \frac{(z_{j,i,t_k} - \mu_{j,i,t_k})^2}{\sigma_{j,i,t_k}^2} \right] + \mathbb{E}_q \left[\sum_{j=1}^{d_z} \frac{(z_{j,i,t_k} - z_{j,i,t_k}^-)^2}{\sigma_{j,i,t_k}^2} \right] \right\} \\ &= \frac{1}{2} \sum_{i=1}^N \left\{ \log \prod_{j=1}^{d_z} \frac{\sigma_{j,i,p,t_k}^2}{\sigma_{j,i,t_k}^2} - d_z + \sum_{j=1}^{d_z} \left[\frac{\sigma_{j,i,t_k}^2}{\sigma_{j,i,p,t_k}^2} + \frac{(\mu_{j,i,t_k} - z_{j,i,t_k}^-)^2}{\sigma_{j,i,p,t_k}^2} \right] \right\}, \end{aligned}$$

where we use the fact that $z_{j,i,t_k} \sim \mathcal{N}(\mu_{j,i,t_k}, \sigma_{j,i,t_k}^2)$ and $\frac{z_{j,i,t_k} - z_{j,i,t_k}^-}{\sigma_{j,i,p,t_k}} \sim \mathcal{N}(\frac{\mu_{j,i,t_k} - z_{j,i,t_k}^-}{\sigma_{j,i,p,t_k}}, \frac{\sigma_{j,i,t_k}^2}{\sigma_{j,i,p,t_k}^2})$.

To estimate $\mathbb{E}_q[\log p(A_{t_k} | z_{t_k})]$, we use Monte Carlo method

$$\mathbb{E}_q[\log p(A_{t_k} | z_{t_k})] \approx \frac{1}{n} \sum_{i=1}^n \log p(A_{t_k} | z_{t_k}^{(i)}).$$

The total loss for all time stamps is

$$\mathcal{L} = \sum_{k=1}^T \mathcal{L}_k.$$

Prediction For a future time t_{T+1} :

$$z_{t_{T+1}}^- = \text{ODESolve}(z_{t_T}, f_\theta, t_T, t_{T+1}).$$

Then we use learnt GCN encoder to embed the feature matrix and get a embedding $H_{t_{T+1}}^{(2)} = \text{GCNEncoder}(X_{t_{T+1}}, I)$. Using $H_{t_{T+1}}^{(2)}$, we construct variational posterior $q(z_{t_{T+1}}) = \prod_{i=1}^N \mathcal{N}(\mu_{i,t_{T+1}}, \sigma_{i,t_{T+1}}^2)$. Predictions follow from

$$\mu_{ij,t_{T+1}} = \text{Decoder}(\mu_{i,t_{T+1}}, \mu_{j,t_{T+1}}).$$

We output either sampled adjacency matrices or their means. When the edge is binary, we output the prediction with a mask matrix by assigning 1 to logits bigger than 0.5 and 0 to the rest.

Table 1: Continuous-Time Latent ODE Graph VAE (training parameter)

Algorithm: Continuous-Time Latent ODE Graph VAE

For iter=1:max

For k=0:K

1. $G_{t_k} = (X_{t_k}, A_{t_k})$, $H_{t_k}^{(2)} = \text{GCNEncoder}(X_{t_k}, A_{t_k})$
 2. $z_{t_k}^- = \text{SolveODE}(z_{t_{k-1}}, f(z(t), t), t_{k-1}, t_k)$
 3. Posterior $z_{t_k} \sim q(z_{t_k}) = \prod_{i=1}^N \mathcal{N}(z_{i,t_k} | \mu_{i,t_k}, \text{diag}(\sigma_{i,t_k}^2))$, $[\mu_{i,t_k}, \sigma_{i,t_k}^2] = \text{MLP}(z_{i,t_k}^-, H_{t_k}^{(2)}(i))$
 4. Decoder $p(A_{t_k} | z_{t_k}) = \prod_{i,j} \mathcal{N}(A_{t_k}(i,j) | \mu_{ij,t_k}, \sigma_d^2)$
 5. Compute gradients $\nabla \tilde{\mathcal{L}}_k$
 6. Update parameter using a gradient ascent method with total gradient $\sum_{k=0}^K \nabla \tilde{\mathcal{L}}_k$
-

Table 2: Continuous-Time Latent ODE Graph VAE (prediction)

Algorithm: Continuous-Time Latent ODE Graph VAE**Given future time stamp t_{K+1} :**

1. $H_{t_{K+1}}^{(2)} = \text{GCNencoder}(X_{t_{K+1}}, I)$
2. $z_{t_{K+1}}^- = \text{SolveODE}(z_{t_K}, f(z(t), t), t_K, t_{K+1})$
3. Posterior $q(z_{t_{K+1}}) = \prod_{i=1}^N \mathcal{N}(z_{i,t_{K+1}} | \mu_{i,t_{K+1}}, \text{diag}(\sigma_{i,t_{K+1}}^2))$, $[z_{i,t_{K+1}}, \sigma_{i,t_{K+1}}^2] = \text{MLP}(z_{i,t_{K+1}}^-, H_{t_{K+1}}^{(2)}(i))$
4. Decoder $p(A_{t_{K+1}} | \mu_{t_{K+1}}) = \prod_{i,j} \mathcal{N}(A_{t_{K+1}}(i,j) | \mu_{ij,t_{K+1}}, \sigma_d^2)$
5. Prediction $A_{t_{K+1}}$ is either
 - i) Sampled from the Decoder or
 - ii) Chosen as the mean

Return $A_{t_{K+1}}$

3.2 Experiment

Since network function is continuous, we have that the latent space is continuous w.r.t time. When strong norm restriction is enforced on the network parameters, the latent space tends to be over smooth which extremely narrows down the expression of the neural network. Furthermore, that the decoder strongly relies on the latent state means the final prediction also lies in a over smoothed space. With all these knowledge, we can conjecture that the algorithm fails to learn complicated hidden dynamic when the network function is over smoothed. In this part, we design two experiments to verify it.

3.2.1 Learning from smoothness

The first experiment aims at testing what the algorithm learns from the dataset. It has a smooth data generation scheme given by

$$\begin{cases} \epsilon^X \in (0,1)^{N \times D} : \epsilon_{ij}^X \sim \mathcal{N}(0,1), \\ \epsilon^P \in (0,1)^{N \times N} : \epsilon_{ij}^P \sim \mathcal{N}(0,1), \\ X : X_{ij} = 0.2 + \sqrt{\epsilon_{ij}}, \\ P : P_{ij} = \frac{1}{1 + \exp\{-0.1(\epsilon_{ij}^P + \epsilon_{ji}^P)\}}, \\ A : A_{ij} = 1, P_{ij} \geq 0.5; 0, P_{ij} < 0.5. \end{cases} \quad (10)$$

The adjacency matrix consists of random variables centering around $\frac{1}{1+\exp(0)} = 0.5$. Hence we choose the edge A_{ij} to be connected if its corresponding logit value P_{ij} is bigger than the mean and vice versa.

Gaussian decoder The algorithm is equipped with a Gaussian decoder. We generate a time series $(t_k, X_k, P_k)_{1 \leq k \leq 20}$ with generator 10, where we set node number, feature number and total time stamps by 5, 10 and 20. The algorithm is trained on the first 19 pieces of the time series for 1000 iterations. Then it predicts the edge probabilities P of the last graph using the second to the last graph.

$$P_{pred} = \begin{bmatrix} 0.5103 & 0.5103 & 0.5103 & 0.5103 & 0.5103 \\ 0.5103 & 0.5103 & 0.5103 & 0.5103 & 0.5103 \\ 0.5103 & 0.5103 & 0.5103 & 0.5103 & 0.5103 \\ 0.5103 & 0.5103 & 0.5103 & 0.5103 & 0.5103 \\ 0.5103 & 0.5103 & 0.5103 & 0.5103 & 0.5103 \end{bmatrix}, P_{true} = \begin{bmatrix} 0.5 & 0.5 & 0.5 & 0.5 & 0.5 \\ 0.5 & 0.5 & 0.5 & 0.5 & 0.5 \\ 0.5 & 0.5 & 0.5 & 0.5 & 0.5 \\ 0.5 & 0.5 & 0.5 & 0.5 & 0.5 \\ 0.5 & 0.5 & 0.5 & 0.5 & 0.5 \end{bmatrix}. \quad (11)$$

The result is shown in (11). We can see that the predicted probability matrix has entries consistently close to 0.5, although with a slight bias. Hence, the algorithm tries to learn the mean value of the distribution that P follows, which is the true dynamic behind the noised samples.

Bernoulli decoder The algorithm is equipped with a Bernoulli decoder. We generate two time series $(t_k^1, X_k^1, A_k^1)_{1 \leq k \leq 20}$ and (t_k^2, X_k^2, A_k^2) . For the first time series, we set node number, feature number and to-

tal time stamps by 5, 1 and 20. For the second one, we set them by 5, 10 and 20. As the same for the two time series, the algorithm is trained on the first 19 pieces of the time series for 1000 iterations. Then it predicts the edge connections A of the last graph using the second to the last graph, and prints the predicted probability matrix as well.

$$P_{pred}^1 = \begin{bmatrix} 0.4606 & 0.4992 & 0.4658 & 0.5057 & 0.4688 \\ 0.4992 & 0.5335 & 0.5038 & 0.5397 & 0.5065 \\ 0.4658 & 0.5038 & 0.4710 & 0.5103 & 0.4741 \\ 0.5057 & 0.5397 & 0.5103 & 0.5457 & 0.5132 \\ 0.4688 & 0.5065 & 0.4741 & 0.5132 & 0.4771 \end{bmatrix}, P_{pred}^2 = \begin{bmatrix} 0.2690 & 0.2740 & 0.6473 & 0.2795 & 0.2771 \\ 0.2740 & 0.2756 & 0.6474 & 0.2809 & 0.2787 \\ 0.6473 & 0.6474 & 0.4332 & 0.6458 & 0.6462 \\ 0.2795 & 0.2809 & 0.6458 & 0.2838 & 0.2827 \\ 0.2771 & 0.2787 & 0.6462 & 0.2827 & 0.2805 \end{bmatrix}. \quad (12)$$

$$P_{true} = \begin{bmatrix} 0.5 & 0.5 & 0.5 & 0.5 & 0.5 \\ 0.5 & 0.5 & 0.5 & 0.5 & 0.5 \\ 0.5 & 0.5 & 0.5 & 0.5 & 0.5 \\ 0.5 & 0.5 & 0.5 & 0.5 & 0.5 \\ 0.5 & 0.5 & 0.5 & 0.5 & 0.5 \end{bmatrix}. \quad (13)$$

The results are shown in (12). Comparing P_{pred}^1 and P_{pred}^2 , we can see that as feature number increases, the noise that the feature brings will inflict heavier impacts on the edge information, and potentially overwhelm it when the sizes are extremely unequal. This matches the behavior of GCN encoder.

Moreover, if we compare the predicted probability matrix in (11) and P_{pred}^2 , we can see that the algorithm struggles to recover the true dynamic from the discretized observations but fails. This implies the discrete step of assigning edge connection is hard to be reversed by the algorithm. The behavior might help prove that the algorithm with norm restriction is hard to dispose non-smooth time series. We will develop another trail to consolidate this statement.

3.2.2 Restricted expression ability

For a neural network restricted to smooth space, we assume discrete trends are too "complicated" to learn by the algorithm. The second experiment bases on a complicatedly designed mixed generator with both a continuous part and a discrete part. We only display the decoder

$$\begin{cases} c : c \in \mathbb{Z}^+ \bigcap (0, C) (\text{Randomized at each time stamp}), \\ M : M_{ij} = 1, c_i = c_j, 0, c_i \neq c_j, \\ l : \lambda M_{ij}, \\ k : k_{ij} = \frac{1}{8}(l_{ij} + l_{ji}) + 4\mathcal{N}(0, 1), \\ A : A_{ij} = \text{Sigmoid}(k_{ij}). \end{cases}$$

If we set $\lambda = 0$ to ignore the discrete term M (Community representation in the code), the adjacency matrix consists of random variables centering around $\frac{1}{8}(l_{ij} + l_{ji})$, which is smaller than 0.5 since l is negative. We first generate a continuous-discrete dynamic mixed time series with 20 time stamps and 1000 iterations, with 4 nodes and 10 features for each. The algorithm uses Gaussian decoder. The training method and prediction are the same as the previous trail. As shown in (14), we find that the prediction is unsatisfactory.

$$A_{mean}^M = \frac{1}{20} \sum_{k=1}^{20} A_{t_k}^M = \begin{bmatrix} 0.9994 & 0.9972 & 0.7906 & 0.0122 \\ 0.9972 & 0.8447 & 0.5514 & 0.9839 \\ 0.7906 & 0.5514 & 0.9983 & 0.9965 \\ 0.0122 & 0.9839 & 0.9965 & 0.9946 \end{bmatrix}, A_{pred}^M = \begin{bmatrix} 0.7837 & 0.7836 & 0.7702 & 0.7732 \\ 0.7836 & 0.7840 & 0.7710 & 0.7738 \\ 0.7702 & 0.7710 & 0.7845 & 0.7824 \\ 0.7732 & 0.7738 & 0.7824 & 0.7847 \end{bmatrix}. \quad (14)$$

However, when the discrete term M is removed, the algorithm gives a better prediction given in (15).

$$A_{mean} = \frac{1}{20} \sum_{k=1}^{20} A_{t_k} = \begin{bmatrix} 0.4222 & 0.6474 & 0.4784 & 0.4391 \\ 0.6474 & 0.4985 & 0.4361 & 0.5528 \\ 0.4784 & 0.4361 & 0.5464 & 0.5548 \\ 0.4391 & 0.5528 & 0.5548 & 0.5021 \end{bmatrix}, A_{pred} = \begin{bmatrix} 0.5033 & 0.5033 & 0.5032 & 0.5033 \\ 0.5033 & 0.5034 & 0.5033 & 0.5033 \\ 0.5032 & 0.5033 & 0.5031 & 0.5032 \\ 0.5033 & 0.5033 & 0.5032 & 0.5033 \end{bmatrix}. \quad (15)$$

Although with a bias, this is consistent with the observation that the edges are all close to 0.5.

In conclusion, this algorithm is good at learning from dynamic with low to moderate complexity, like learning from data generated by a continuous distribution. When the true data generation scheme is a mixture of continuous and discrete parts, the model usually fails to recover the continuous part from the chaos.

As an additional discussion (experiment is not shown here), when the true smooth dynamic is complicated but learnable, the algorithm needs longer time series and larger iterations to train. This shows that this algorithm is data-hunger, consistent with the intrinsic nature of neural ODE based models.

References

- [1] Ricky T. Q. Chen et al. *Neural Ordinary Differential Equations*. 2019. arXiv: [1806.07366](#) [cs.LG]. URL: <https://arxiv.org/abs/1806.07366>.
- [2] Diederik P Kingma and Max Welling. *Auto-Encoding Variational Bayes*. 2022. arXiv: [1312.6114](#) [stat.ML]. URL: <https://arxiv.org/abs/1312.6114>.